Pipeline Documentation

The `Pipeline.py` script is designed to perform a series of data processing tasks specifically tailored for predictive modeling in a winery context. The script encompasses various stages such as data loading, cleaning, feature selection, data splitting, hyperparameter tuning, model training, and evaluation.

Below is a detailed explanation of each component within the script:

Functions Overview

Function	Description
`load_data`	Loads data from a CSV file into a pandas DataFrame.
`clean_data`	Performs data cleaning operations such as filling missing values, replacing values, and converting data types.
`select_features`	Uses `SelectKBest` with ANOVA F-test to select features for the model.
`split_data`	Splits the dataset into training and testing sets and exports the test set.
`tune_hyperparameters`	Uses `GridSearchCV` to find the best model parameters for different regression algorithms.
`evaluate_model`	Evaluates the trained model using metrics like MSE, RMSE, R2 score, and MAPE.
`process`	Main function that orchestrates the execution of all the steps in the pipeline.

Data Loading

The `load_data` function takes a file path as input and returns a DataFrame after loading the data from the specified CSV file.

```
def load_data(file_path): data = pd.read_csv(file_path) return data
```

Data Cleaning

`clean_data` function removes unnecessary columns, fills missing values with the most frequent or median values, and encodes categorical variables as integers.

```
def clean_data(data): # Various cleaning operations... return data
```

Feature Selection

The `select_features` function performs feature selection using ANOVA F-test to identify the most important features.

```
def select_features(X, y, k='all'): # Feature selection process... return X_new, selected_features
```

Splitting Data

`split_data` function divides the data into training and test sets and saves the test set to a CSV file.

```
def split_data(X, y, test_size=0.3): # Data splitting process... return X_train, X_test, y_train,
y_test, test_csv
```

Hyperparameter Tuning

`tune_hyperparameters` function performs a grid search to find the best hyperparameters for either `RandomForestRegressor` or `GradientBoostingRegressor`.

```
def tune_hyperparameters(X_train, y_train, reg_algo): # Hyperparameter tuning process... return best_model, best_params
```

Model Evaluation

The `evaluate_model` function calculates several metrics to assess the performance of the trained model.

```
def evaluate_model(model, X_test, y_test): # Model evaluation process... return MSE, RMSE, r2, MAPE, y_pred
```

Process Function

The `process` function is the main driver of the pipeline, sequentially calling the previous functions to execute the end-to-end process.

```
def process(file_path): # Orchestrates the data processing pipeline... return tuned_mse, best_params
```

Example Usage

To utilize this pipeline, one would call the `process` function with the appropriate file path to the dataset:

```
tuned_mse, best_params = process('path_to_dataset.csv')
```

Metrics Considered

The metrics considered for model evaluation in the project include:

- Mean squared error (MSE)
- Root mean squared error (RMSE)
- R2 score
- Mean absolute percentage error (MAPE) These metrics are commonly used in regression tasks to evaluate the performance of the models. The `MSE` and `RMSE` measure the average squared and square root of the differences between the predicted and actual values, respectively. The `R2 score` measures the proportion of the variance in the dependent variable that is predictable from the independent variables. The `MAPE` measures the percentage difference between the predicted and actual values. The evaluation of the models using these metrics will allow for a comparison of the performance of at least two regression models in terms of accuracy. The best-performing model can then be selected for generating predictions for the test sample.

Model Comparision

Model Comparison

This section compares the performance of two regression models: Random Forest Regressor and Gradient Boosting Regressor. The comparison is based on the following metrics:

- Mean Squared Error (MSE): A measure of the average squared difference between the predicted and actual values.
- Root Mean Squared Error (RMSE): The square root of MSE, which is a more intuitive measure of error.
- Mean Absolute Percentage Error (MAPE): A measure of the average percentage error, which is useful for comparing models on different scales.
- R-squared (R2): A measure of how well the model explains the variation in the data. The following table shows the results for both models:

Metric	Random Forest Regressor	Gradient Boosting Regressor
Mean Squared Error (MSE)	4027.888190622582	4586.013589679178
Root Mean Squared Error (RMSE)	63.46564575124547	67.72011215052126
Mean Absolute Percentage Error (MAPE)	0.17415292832508358	0.24403451439421447
R-squared (R2)	0.8122228276995535	0.7862034338474846

Based on the above results, the Random Forest Regressor model outperforms the Gradient Boosting Regressor model on all metrics. The Random Forest Regressor model has a lower MSE, RMSE, and MAPE, and a higher R2 score.

The Random Forest Regressor model is the better model for this regression task. It has lower error and higher predictive power than the Gradient Boosting Regressor model.

Wine CLI Documentation

`Wine_cli.py` is a command line interface (CLI) Python script designed to interact with the Wine data processing pipeline. It utilizes the `argparse` library to handle command line arguments and integrates with a module named `Pipeline`, which is expected to contain a `process` function.

Overview

The script is intended to provide a simple command line utility for users to process a CSV file containing Wine data. It allows users to specify the path to the input CSV file as a command line argument.

Usage

To use the script, run it from the command line and provide the path to the input CSV file as follows:

```
python Wine_cli.py input_csv.csv
```

Replace `input_csv.csv` with the actual path to your input CSV file.

Code Explanation

Here's a brief explanation of each part of the `Wine_cli.py` script:

Imports

```
import argparse from Pipeline import process
```

- `argparse`: The script uses this built-in Python library to parse command line arguments.

- `process`: Imports the `process` function from the `Pipeline` module, which is expected to handle the wine data processing logic.

Argument Parser Setup

```
parser = argparse.ArgumentParser() parser.add_argument('input_csv', help='Insert the filepath of the input csv')
```

- An `ArgumentParser` object is created to handle the command line input.
- The `add_argument` method defines a required positional argument `input_csv`. It is expected to be the filepath of the input CSV file which contains the Wine data to be processed.

Read Arguments

```
args = parser.parse_args()
```

- The `parse_args` method reads the arguments provided to the script from the command line.

Process the CSV file

```
new_data = process(args.input_csv)
```

- The `process` function from the `Pipeline` module is called with the input CSV file path as its argument. The function is expected to perform the necessary data processing and return the processed data.

Requirements

To run `Wine_cli.py`, you need:

- Python 3.x installed on your system.
- A `Pipeline` module with a `process` function implemented.
- An input CSV file containing the data you wish to process.

Conclusion

The project successfully addressed the task of estimating wine prices from multivariate inputs using regression models. Key achievements and findings from the project include:

- Development of a robust data pipeline for data loading, cleaning, feature selection, model training, and evaluation.
- Utilization of two regression models, Random Forest and Gradient Boosting Regressor, for the wine price estimation task.
- Comparison of the models based on performance metrics, such as Mean Squared Error, Root Mean Squared Error, R2 Score, and Mean Absolute Percentage Error.
- Selection of the best-performing model for generating predictions, which can be used for practical applications in the wine industry.

The project successfully estimated wine prices using regression models, specifically Random Forest and Gradient Boosting Regressor. The Random Forest regression algorithm generated an estimate of 80% accuracy, which was confirmed by the R2 score. The comparison of the two models based on performance metrics revealed that the Random Forest model outperformed the Gradient Boosting Regressor model in terms of MSE,

RMSE, and MAPE. Therefore, the Random Forest model was selected for generating predictions for the test sample.

The project's documentation, including the Python scripts and the model comparison results, provides a valuable framework for future regression tasks and serves as a reference for implementing similar machine learning solutions. The project's success in achieving the stated objectives demonstrates the effectiveness of the applied machine learning techniques and the potential for real-world applications in the wine industry.